

A Beginner's Guide to

Agile Methodology

...and every Agile term used in the Expense Tracker Enterprise project

This guide has two parts. Part 1 explains Agile and Scrum from first principles — no prior experience assumed. Part 2 is a glossary that defines every Agile term already used in this project's **BACKLOG.md** and **SPRINT_PLAN.md**, with a concrete example from this project next to each one.

Companion documents: BACKLOG.md · SPRINT_PLAN.md · HLD.md · LLD.md · DATABASE_DESIGN.md
Expense Tracker Enterprise — Project Documentation

Part 1 — Understanding Agile

1. What Problem Is Agile Solving?

Before Agile, most software was built using a **Waterfall** approach: gather every requirement up front, design the entire system, build all of it, then test all of it, then release. This works fine when requirements are fully known in advance and never change — which, for most real software, is rarely true. Requirements shift as users see working software; a plan written six months before anything is demoed is often wrong by the time it's finished.

Agile is a response to that problem: instead of one big plan-build-release cycle, the work is broken into small pieces that are each planned, built, and demoed in short, repeating cycles. This surfaces mistakes and changing needs early — while they're cheap to fix — instead of at the very end, when they're expensive.

2. The Agile Manifesto

In 2001, a group of software practitioners published the Agile Manifesto — the founding statement of the whole movement. It's short by design: four value statements, each preferring one thing over another (not rejecting the second item — just weighting the first higher).

We value...	...over...
Individuals and interactions	Processes and tools
Working software	Comprehensive documentation
Customer collaboration	Contract negotiation
Responding to change	Following a plan

"That is, while there is value in the items on the right, we value the items on the left more." — Agile Manifesto, 2001. (This declaration may be freely copied in any form, but only in its entirety through this notice.)

3. Agile vs. Waterfall, Side by Side

	Waterfall	Agile
Planning	All up front, one time	Continuous, revisited every sprint
Delivery	One release at the end	Small working increments, repeatedly
Change	Expensive, resisted	Expected and welcomed
Feedback	Mostly at the end	Every 1–4 weeks (every sprint)
Risk	Discovered late	Surfaced early, in small batches

4. Scrum: The Specific Flavor of Agile Used Here

"Agile" is a set of values and principles, not a step-by-step process — several concrete **frameworks** implement it (Scrum, Kanban, XP, and others). This project's SPRINT_PLAN.md uses **Scrum**, the most widely used one. Scrum organizes work into fixed-length iterations called **Sprints** (2 weeks in this project), and defines a small set of roles, ceremonies, and artifacts around them.

Scrum has three roles:

- **Product Owner** — decides *what* gets built and in what order (owns and prioritizes the backlog).
- **Scrum Master** — protects the process: removes blockers, facilitates ceremonies, keeps the team honest to Scrum's rules.
- **Development Team** — decides *how* to build it and does the building.

On a 1–3 person portfolio project like this one, a single person typically wears all three hats — the roles still matter conceptually (someone has to decide priority, someone has to protect focus, someone has to build), they just aren't separate people.

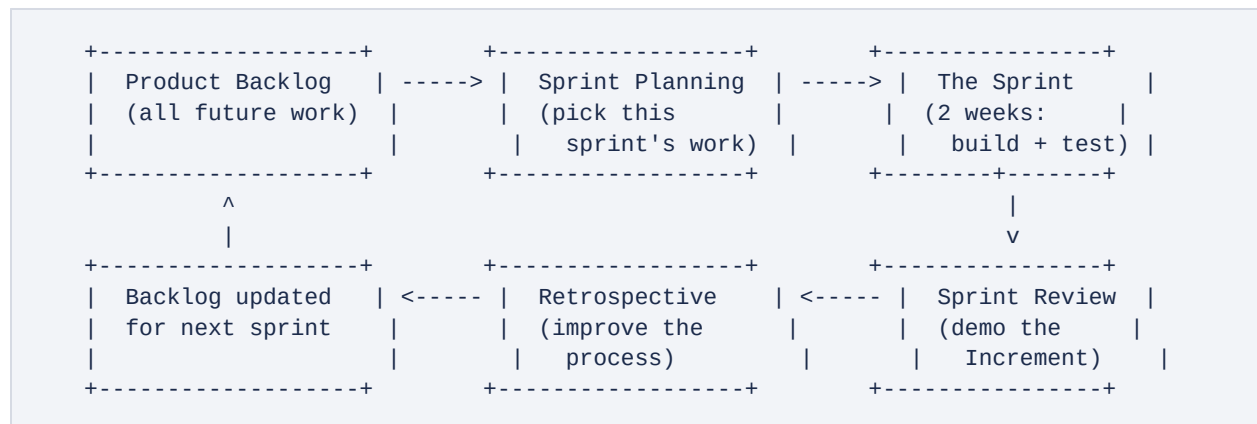
Scrum has three artifacts:

- **Product Backlog** — the full ranked list of everything that could be built. This project's BACKLOG.md.
- **Sprint Backlog** — the slice of the Product Backlog pulled into the current sprint.
- **Increment** — the working, tested software produced by a sprint — always potentially shippable.

Scrum has five events (ceremonies):

- **The Sprint** itself — the fixed time-box everything else happens inside of.
- **Sprint Planning** — at the start: decide what goes into this sprint's backlog.
- **Daily Scrum / Standup** — a short daily sync on progress and blockers.
- **Sprint Review** — at the end: demo the working increment.
- **Sprint Retrospective** — at the end: reflect on process and improve for next time.

5. The Scrum Cycle



The Scrum cycle: this loop repeats every sprint (every 2 weeks in this project) for the life of the product.

This is the exact loop SPRINT_PLAN.md's 8 sprints run: each one has a stated goal (Sprint Planning's output), a fixed 2-week box of work, and closes with a review/demo and an implied retrospective before the next sprint starts.

Part 2 — Glossary: Every Agile Term Used in This Project

Each entry below defines the term in general Agile usage, then shows exactly where and how it's used in this project's own BACKLOG.md / SPRINT_PLAN.md — so you can flip between this guide and the real documents and see the concept in place.

Roles

Product Owner

The person who decides what gets built and in what priority order — owns the backlog and represents the users'/business's interests.

In this project: Not a separate named role in the docs — on this small-team project, the person prioritizing BACKLOG.md's MoSCoW ranking is acting as Product Owner.

Scrum Master

The person who facilitates Scrum ceremonies, removes blockers, and keeps the team following the process — a process/coaching role, not a technical lead role.

In this project: SPRINT_PLAN.md's "Ceremonies" section (standups, review, retro) describes this role's responsibilities without naming a separate person, appropriate for a 1–3 developer team.

Development Team

The people who actually design, build, and test the product each sprint.

In this project: The "1–3 developers" assumption stated at the top of SPRINT_PLAN.md.

Artifacts

Backlog / Product Backlog

The complete, ranked list of everything that could be built for the product — never "done," constantly reordered as priorities change.

In this project: BACKLOG.md — all 37 user stories across 8 epics.

Sprint Backlog

The subset of the Product Backlog the team commits to finishing in the current sprint.

In this project: Each sprint's story table in SPRINT_PLAN.md, e.g. Sprint 1's five stories (US-001, US-002, US-003, US-007, US-005).

Increment

The sum of everything that meets the Definition of Done by the end of a sprint — a working piece of the product, not just merged code.

In this project: Implied by every sprint's goal statement ("ending with something runnable") and made explicit at Sprint 5 ("MVP release candidate") and Sprint 8 ("Full V1.1 release").

Backlog & Story Terms

Epic

A large body of related work, too big to build in one sprint, broken down into smaller user stories.

In this project: BACKLOG.md's 8 epics (e.g. "Epic 05 — Budget Management"), each mapped directly to one of the architecture's bounded contexts (apps/budgets, apps/expenses, etc.).

User Story

A small, independently valuable slice of work, written from the user's point of view: “As a [user], I want [goal] so that [benefit].” Deliberately focused on outcome, not implementation detail.

In this project: Every US-### entry in BACKLOG.md, e.g. US-014: “As a user, I want to create an expense with title, amount, category... so that I can record my spending.”

Enabler Story

A story with no direct user-facing behavior — purely technical foundation work needed to unblock later stories. Still written and sized like a normal story so it's tracked, not hidden.

In this project: US-001 (Project Scaffolding) and US-002 (Common Package) are explicitly labeled “(Enabler)” in BACKLOG.md, phrased as “As a developer...” instead of “As a user...”

Acceptance Criteria

The specific, testable conditions a story must satisfy to be considered correctly built. Turns a fuzzy goal into something a reviewer can actually check off.

In this project: The bullet list under every story, e.g. US-015's criteria that a non-existent or another user's expense ID must return 404, not 403 — precise enough to write a test against.

Dependency

A story that cannot start until another story is finished, because it needs something the earlier story produces (an API, a table, a working auth flow).

In this project: Every story in BACKLOG.md lists its Dependencies field — e.g. US-025 (Budget Threshold Alerts) depends on US-014 (Create Expense) and US-024 (View Budget Status). Dependencies are exactly what SPRINT_PLAN.md's sprint ordering was built to respect.

Estimation & Prioritization Terms

Story Points

A relative measure of effort/complexity/risk for a story — deliberately not hours, because “how big does this feel compared to that other story” is easier and more stable to estimate than “how many hours exactly.”

In this project: Every story's Size field, e.g. US-014 — Size: 5.

Fibonacci Sizing

The specific number sequence (1, 2, 3, 5, 8, 13...) commonly used for story points. The growing gaps between numbers force real decisions — you can't waste time debating “5 vs. 6,” because 6 isn't an option.

In this project: BACKLOG.md's sizing scale note: “Fibonacci story points (1, 2, 3, 5, 8)... 8 is reserved for genuinely substantial stories.”

T-Shirt Sizing

An alternative, coarser estimation technique using sizes like S/M/L/XL instead of numbers — faster but less precise than story points. Mentioned as an option for this kind of planning, not the one actually chosen here.

In this project: Not used in this project's docs — Fibonacci story points were used instead, for finer-grained prioritization within each epic.

MoSCoW

A prioritization technique that sorts every item into one of four buckets: **M**ust have, **S**hould have, **C**ould have, **W**on't have (this time). Forces explicit trade-off decisions instead of treating everything as equally urgent.

In this project: BACKLOG.md's MoSCoW Summary table: 21 Must stories (84 points) define the MVP, 9 Should and 7 Could stories (47 points) make up the later V1.1 release.

Velocity

How many story points a team actually completes per sprint, measured after the fact. Used to forecast how many future sprints are needed for the remaining backlog — a measured number, not a target to hit.

In this project: SPRINT_PLAN.md's Velocity Summary table (23, 20, 17, 16... points per sprint), with an explicit note to "re-baseline this table after Sprint 1–2 actuals."

Delivery & Planning Terms

Sprint

A fixed-length time-box (commonly 1–4 weeks) in which a set of backlog items is planned, built, tested, and demoed. The heartbeat of Scrum — everything else in the framework happens on this cadence.

In this project: 2-week sprints, 8 total, in SPRINT_PLAN.md.

Sprint Goal

A single sentence describing the purpose of a sprint — what should be true and demoable by the end of it. Gives the team a shared "why," not just a ticket list.

In this project: Every sprint section in SPRINT_PLAN.md opens with a Goal line, e.g. Sprint 4's: "Users can set a monthly budget and see accurate, race-safe spend/remaining/threshold-alert status."

Sprint Planning

The ceremony at the start of a sprint where the team selects which backlog items become the Sprint Backlog, guided by priority and the Definition of Ready.

In this project: Described in SPRINT_PLAN.md's Ceremonies section; the resulting output is each sprint's story table.

Daily Standup / Daily Scrum

A short daily check-in (traditionally 15 minutes, done standing up to keep it brief) covering what was done, what's next, and any blockers.

In this project: SPRINT_PLAN.md notes an async written update is sufficient for a team under 3 people — the ceremony's value (surfacing blockers early) doesn't require a live meeting at this scale.

Sprint Review / Demo

The ceremony at the end of a sprint where the actual working Increment is shown — not described, not slides, the running software.

In this project: SPRINT_PLAN.md's Ceremonies section: "demo working software against the sprint goal — every sprint... ends with something runnable, not just merged code."

Sprint Retrospective

The ceremony at the end of a sprint focused on the team's process itself: what worked, what didn't, one concrete change to try next sprint.

In this project: SPRINT_PLAN.md's Ceremonies section, listed as the closing ritual of every sprint.

Definition of Ready (DoR)

The checklist a backlog item must pass before it's allowed into a sprint — prevents starting work that's too vague or blocked to finish.

In this project: SPRINT_PLAN.md's Definition of Ready section: acceptance criteria written, dependencies sequenced, sized, no open blocking questions.

Definition of Done (DoD)

The checklist every story must pass to count as actually finished — not “code written,” but tested, reviewed, documented, and merged.

In this project: SPRINT_PLAN.md's Definition of Done section: business logic in the correct layer, unit tests passing, Swagger docs updated, code reviewed, merged.

MVP (Minimum Viable Product)

The smallest version of the product that is still genuinely complete and useful — not a rough prototype, but every “Must” requirement with nothing extra.

In this project: SPRINT_PLAN.md's Release Plan: all 21 Must stories, shipping as “MVP release candidate” at the end of Sprint 5.

Release Plan

The plan for what ships, and when, across multiple sprints — groups sprints into meaningful shippable milestones rather than treating every sprint as an isolated unit.

In this project: SPRINT_PLAN.md's Release Plan table: MVP at Sprints 1–5, V1.1 at Sprints 6–8.

Hardening Sprint / Buffer

Time deliberately reserved (inside or as a whole sprint) for integration testing and bug-fixing rather than new features — used when multiple previously-separate pieces are about to be exercised together for the first time.

In this project: Sprint 5 explicitly budgets an ~8-point buffer for “cross-module manual QA... MVP demo prep,” reasoned as the first sprint where every module meets through the Dashboard facade.

Worked Example: One Story, Every Term Applied

To see these terms working together rather than in isolation, here is US-025 from BACKLOG.md, annotated end to end.

US-025: Budget Threshold Alerts

Epic: 05 — Budget Management **Priority (MoSCoW):** **Must** **Story Points:** 5

***User Story:** “As a user, I want to be alerted when my spending crosses 80%, 90%, and 100% of my budget so that I can course-correct before overspending.”*

Acceptance Criteria (what makes this Done, not just coded):

- Crossing each threshold fires exactly one alert for that tier per budget period.
- Crossing 100% directly from below 80% still fires only the highest-tier alert, not all three.
- The Observer pattern from the architecture docs is used, not ad hoc checks scattered in views.

Dependencies: US-014 (Create Expense) and US-024 (View Budget Status) — this is exactly why the sprint plan schedules Expense CRUD (Sprint 2) and basic Budget viewing (Sprint 4, earlier in the same sprint) before this story.

Where it lives in the Sprint Backlog: Sprint 4 — “Budgets & Threshold Safety,” whose Sprint Goal is exactly this capability, alongside its sibling US-026 (the race-safety hardening for the same feature).

Contribution to Velocity: its 5 points are part of Sprint 4's 16-point total in the Velocity Summary table.

Path to Increment: once implemented, tested against every Acceptance Criterion, reviewed, and merged (Definition of Done), it becomes part of the Sprint 4 Increment — and because it's a “Must” story, it's also part of what makes the Sprint 5 MVP release candidate complete.

Suggested next step: read BACKLOG.md and SPRINT_PLAN.md side by side with this guide open — every term here should now point to something concrete you can find in those files.